

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе №2

«Таблицы»

Выполнил:

студент группы 3824Б1ФИ2

Большаков В.В.

Проверил:

Ивлев А.Д.

Нижний Новгород
2026

Содержание

Введение.....	3
1. Постановка задачи.....	4
2. Руководство пользователя	5
3. Руководство программиста	6
3.1. Описание структуры программы	7
3.2. Описание структур данных.....	9
3.3. Описание алгоритмов.....	16
4. Результаты экспериментов и тестирования.....	24
Заключение.....	28

Введение

Краткое описание предметной области и задачи

Описание предметной области: предметом исследования являются такие структуры данных как таблицы, которые представлены тремя типами:

- Неупорядоченная таблица (на векторе)
- Упорядоченная таблица на сбалансированном бинарном дереве поиска (AVL дерево)
- Хэш-таблица

Описание задачи: реализация программы способной работать с таблицами данных типов, интеграция данных таблиц в единое консольное приложение, а также проведение экспериментов для оценки и сравнения эффективности операций с данными при увеличении объёма хранимой информации. Особое внимание уделяется подсчёту количества выполненных операций и логированию результатов, что позволяет объективно оценить производительность каждой структуры в различных сценариях использования.

Используемый инструмент для выполнения задачи: язык программирования C++, фреймворк **Google Test** для написания автоматического набора тестов, направленных на проверку корректности разработанной функциональности.

Сферы применения реализованной программы: реализованные в программе структуры данных могут быть применены в системах управления базами данных (хранение, добавление, извлечение данных).

1. Постановка задачи

Постановка задачи: разработать программу, которая предоставляет возможность хранить данные в таблицах трёх типов с подсчётом количества произведённых операций с момента создания (неупорядоченная таблица (на векторе), упорядоченная таблица на сбалансированном бинарном дереве поиска (AVL дерево), хэш-таблица).

Таблицы должны поддерживать хранение произвольных данных, но в качестве данных в пользовательском консольном приложении, при тестировании и проведении экспериментов используются полиномы из первой лабораторной работы («Арифметические операции с полиномами»)

Работоспособность программы проверяется с помощью автоматических тестов Google Test. Также необходимо реализовать пользовательское консольное приложение и необходимый функционал для проведения экспериментов.

Условия и ограничения:

- В консольном приложении и при проведении экспериментов работа происходит сразу с таблицами всех типов
- Ключ – строка, сравнение лексикографическое.
- Поддерживаемые операции:
 - Вставить данные в таблицу с заданием для них некоторого наименования (ключа)
 - Удалить данные по заданному ключу (данные удаляются вместе с ключом)
 - Найти данные по заданному ключу (пользователь получает данные, но они остаются в таблице)
 - Извлечь данные по заданному ключу (пользователь получает данные, но они удаляются из таблицы (вместе с соответствующим ключом))
- Хэш-таблица реализуется с одним из способов борьбы с коллизиями (в данной работе реализуется способ: «Повторное хэширование»).
- В пользовательском консольном приложении должна быть возможность производить операции с полиномами, а также размещать полученный результат в таблицу.
- Во время работы пользовательского консольного приложения происходит логирование количества произведённых операций. Т.е. программа считает и записывает в файл, какое действие произошло и сколько операций потребовалось на него у разных типов таблиц. После завершения работы в файл добавляется общее число операций для каждого действия и в целом у разных типов таблиц

2. Руководство пользователя

В качестве основы пользовательского консольного приложения использовалось консольное приложение из лабораторной работы «Арифметические операции с полиномами».

При включении программы пользователю выводится сообщение с просьбой выбрать режим начала работы программы:

1. Создать `polinom_1` и `polinom_2` (если пользователь захочет создать пустой полином, то достаточно нажать “enter”)
0. Завершить работу программы (при этом в файл “Logging operation.txt” будет записано общее число операций для каждого действия у всех таблиц)
- 1. Очистить файл “Logging operation.txt” (при этом программа завершит работу)

Режим -1: при работе с таблицами в файл “Logging operation.txt” записывается количество произведенных действий при выполнении определенной операции (см. пункт «Постановка задачи»: «Поддерживаемые операции»). Если программа запускается не первый раз, то данный файл не будет пустым (если режим -1 не применялся)

После создания полиномов пользователю предлагается выбрать из списка возможную операцию:

- операции 0 – 10 описаны в отчёте для лабораторной работы №1
- 11 – работа с таблицами.

Если пользователь выбрал режим **11**, то ему будет предложено выбрать одну из следующих операций:

0. Удалить запись
1. Вставить полином в таблицу
*имеется введу `polinom_1` или `polinom_2`
2. Добавить новый полином в таблицу
*пользователю придется создать новый полином
3. Найти данные записи (получить полином по заданному ключу)
*пользователь получит полином, при этом данные из таблицы не исчезнут
4. Извлечь запись (пользователь получит полином, но данные из таблицы будут удалены)
5. Завершить работу (вернуться к операциям с полиномами)

Если при выборе режима допущена ошибка, то ввод запрашивается до тех пор, пока пользователь не введет корректные данные.

При выборе операций: **0 – 4** – пользователь должен будет ввести ключ записи. Единственное ограничение: ключ не может быть пустой строкой (допускаются любые символы).

При выборе операций **3,4** пользователь получает данные (полином) из таблицы. При этом данные из таблицы заменяют один из полиномов: `polinom_1` или `polinom_2`. Пользователю будет предложено выбрать режим:

1. Заменить `polinom_1`
2. Заменить `polinom_2`
0. Отменить операцию

При выборе операции **1** пользователю нужно будет выбрать режим:

1. `Polinom_1` будет добавлен в таблицу
2. `Polinom_2` будет добавлен в таблицу
3. Результат арифметической операции между `polinom_1` и `polinom_2` будет добавлен в таблицу (если не производилось арифметической операции между данными полиномами, то в таблицу будет добавлен пустой полином)

3. Руководство программиста

3.1. Описание структуры программы

Программа состоит из двух проектов: Lab№2.4 и Project1.

Проект Lab№1.4 содержит файл **test.cpp**, который содержит в себе все тесты Google Test. Данный проект предназначен в первую очередь для проверки работоспособности классов проекта Project1. Кроме тестов в файле test.cpp содержится функция main(), в которой происходит генерация данных для экспериментов и вывод результатов в консоль (изначально данная функция задокументирована)

Перейдем к проекту **Project1**. В данном проекте 4 основных файла: **Source.cpp**, **Experiments.h**, **Polinom.h**, **Tables.h**. В файле Polinom.h содержится реализация класса Polinom из лабораторной работы №1.

Файл **Tables.h** содержит реализацию всех типов таблиц. Каждая из таблиц реализована в виде шаблонного класса. Таблицы:

- **unordered_table** – неупорядоченная таблица
- **AVLtable** – упорядоченная таблица
- **HashTable** – хэш-таблица

Также данный файл содержит структуру **Record** (данная структура используется в функциях для проведения экспериментов), класс **Logging operations** (предназначен для записи количества произведенных операций для конкретного действия (поиск, вставка и т.д.) в файл для каждой таблицы). Logging operations – шаблонный класс (т.к. таблицы – это шаблонные классы)

Файл **Experiments.h** содержит реализацию функций, необходимых для проведения экспериментов. Функции:

- generate_key – создает уникальный ключ вида «key_0001, key_0002 ...»
- generate_degr – создает случайную степень (значения от 0 до 999)
- generate_pol – использует две предыдущие функции для создания полинома
- run_insert_experiment – производит подсчет количества операций и замер времени, которое потребовалось на вставку данных у разных таблиц.
- run_delete_experiment - производит подсчет количества операций и замер времени, которое потребовалось на удаление данных у разных таблиц.
- run_find_experiment – производит подсчет количества операций и замер времени, которое потребовалось на поиск данных у разных таблиц.

Методы run возвращают время и количество операций, которые потребовались на соответствующее действие у всех таблиц.

Файл **Source.cpp** содержит реализацию пользовательского консольного приложения. Т.к. данное консольное приложение основано на консольном приложении из лабораторной работы №1, то оно содержит все старые функции, описанные в отчёте к первой лабораторной работе (некоторые из них были немного изменены). Новые

функции:

- **input_key** – позволяет пользователю ввести ключ для записи в таблице.
- **Choice_mode** - позволяет пользователю выбрать один из перечисленных режимов.
(используется в функциях, которые запрашивают у пользователя выбор режима)
- **choice_replace_polinom** – используется, когда пользователь должен выбрать, какой из полиномов (polinom_1 или polinom_2) заменить на данные из таблицы.
- **choice_insert_polinom** - используется, когда пользователь должен выбрать, какой из полиномов нужно вставить в таблицу.
- **work_witch_table** – функция, в которой происходит основная работа с таблицами
(позволяет пользователю выбрать, какую операцию нужно произвести с таблицами;
использует предыдущие функции)

3.2. Описание структур данных

Файл **Tables.h**

Подключенные библиотеки: **iostream** (для операторов ввода и вывода), **vector** (для работы с классом `std::vector`), **fstream** (работа с файловыми потоками), **stack** (используется в реализации класса `AVLtable`), **“Polinom.h”** (данный класс используется в объявлениях дружественных функций для экспериментов)

Описание структуры **Record**:

Данная структура необходима для проведения экспериментов, используется в соответствующих дружественных функциях.

Описание полей:

- `key` – объект класса `std::string` (ключ к данным записи в таблице)
- `polinom` – объект класса `Polinom` (данные, доступ к которым осуществляется с помощью ключа `key`)

Описание методов:

- **Record()** – конструктор по умолчанию
- **Record(const std::string& k, const Polinom& v)** – конструктор инициализации. Принимает ссылку на константный объект класса `std::string`, ссылку на константный объект класса `Polinom`. Присваивает полям структуры переданные параметры.

Определение таблиц: таблица – набор записей, состоящих из полей, одно из которых является ключом, а остальные образуют тело записи.

Соответственно во всех типах таблиц, есть структура, содержащая ключ и данные.

Описание класса **unordered_table**:

*некоторые моменты, связанные с логированием количества произведенных операций и экспериментами, будут опущены, т.к. они одинаковы для всех типов таблиц (о них рассказывается после описания таблиц)

Данный класс является шаблонным и представляет собой реализацию неупорядоченной таблицы.

Описание полей (спецификатор доступа **private**):

- Структура `record`. Содержит два поля:
 - `key` – объект класса `std::string`
 - `value` – представлено шаблонным типом `Tvalue` (данные записи, доступ к которым осуществляется по ключу `key`)
- `table` – объект класса `std::vector`, который хранит данные типа `record`
- `count_operations` – представлено типом `size_t`. Используется для подсчета количества операций при выполнении определенного действия (поиск, вставка, удаление, извлечение)

Описание методов (спецификатор доступа **public**):

unordered_table() = default – конструктор по умолчанию.

GetSize(): константный метод. Возвращает размер поля table (т.е. возвращает тип `size_t`)

Delete(const std::string& key) – принимает ссылку на константный объект класса `std::string`. Возвращает тип `void`. Используется для удаления записи из таблицы по переданному ключу.

Find(const std::string& key) – принимает ссылку на константный объект класса `std::string`. Возвращает указатель на тип `Tvalue`. Данный метод используется для нахождения данных по переданному ключу. Возвращение указателя позволяет изменять данные (ключ при этом не поменяется; также можно разыменовать указатель и работать только с данными). Если запись с данным ключом не найдена в таблице, то возвращается `nullptr`.

Insert(const std::string& key, const Tvalue& value) – принимает ссылку на константный объект класса `std::string`, ссылку на константу типа `Tvalue`. Данный метод используется для вставки записи в таблицу (`value` – данные, `key` – ключ). Возвращает тип `void`.

Extract(const std::string& key) – принимает ссылку на константный объект класса `std::string`. Возвращает объект типа `Tvalue`. Извлекает запись из таблицы, т.е. при использовании данного метода пользователь получает данные записи (передавая соответствующий ключ), но после данная запись будет удалена из таблицы.

Описание класса **AVLtable**:

Данный класс является шаблонным и представляет собой реализацию упорядоченной таблицы на АВЛ дереве.

Описание полей (спецификатор доступа **private**):

- Структура **Node**. Поля:
 - `key` – объект класса `std::string` (сравнение ключей лексикографическое)
 - `data` – данные (шаблонного типа `Tvalue`)
 - `left` – указатель на тип `Node` (указатель на левого потомка данной вершины)
 - `right` – указатель на тип `Node` (указатель на правого потомка данной вершины)
 - `height` – представлено типом `int` (разница между высотой правого поддерева и высотой левого поддерева данной вершины)
 - ❖ **Node(const std::string& k, const Tvalue& d)** – конструктор инициализации. Принимает ссылку на константный объект класса `std::string`, ссылку на константный объект типа `Tvalue`. Присваивает полю `key` параметр `k`, полю `data` параметр `d`, поля `left` и `right` получают значение `nullptr`, поле `height` получает значение 0.
- `root` – указатель на тип `Node` (указатель на корень дерева)

- `size` – размер таблицы (дерева). Имеет тип `size_t`
- `count_operations` – представлено типом `size_t` (использование аналогично классу `unordered_table`)

Описание методов (спецификатор доступа **private**):

getHeight(Node* node) – принимает указатель на тип `Node`. Возвращает высоту данного узла, т.е. значение поля `height` (если `node` равен `nullptr`, то возвращается -1)

NewHeight(Node* node) – принимает указатель на тип `Node`. Обновляет поле `height` для переданной вершины `node`. Возвращает тип `void`.

rightRotate(Node* node) – принимает указатель на тип `Node`. Используется в случае, если баланс переданной вершины нарушен (его нужно исправить). Возвращает указатель на тип `Node`.

leftRotate(Node* node) – принимает указатель на тип `Node`. Возвращает указатель на тип `Node`. Аналогично `rightRotate` используется при нарушении баланса вершины.

*допустимые значения баланса узла: -1, 0, +1. В зависимости от нарушения применяется или `leftRotate` или `rightRotate`.

Balancing(std::stack<Node*>& path) – принимает ссылку на объект класса `std::stack`, содержащего указатели на тип `Node`. Используется в методах `Delete` (удаление вершины), `Insert` (вставка вершины), т.к. после вставки/удаления баланс какой-либо вершины может быть нарушен (исправление баланса). Возвращает тип `void`.

print_node(std::ostream& out, Node* node, size_t& count) – принимает ссылку на объект `std::ostream`, указатель на тип `Node`, ссылку на тип `size_t`. Является константным методом. Используется в реализации оператора вывода.

Описание методов (спецификатор доступа **public**):

AVLtable() – конструктор по умолчанию. Присваивает полю `size` значение 0, полю `root` значение `nullptr`.

Назначение методов `Delete`, `Find`, `Insert`, `Extract` такое же, как и в классе `unordered_table` (принимаемые параметры и возвращаемые значения точно такие же).

GetSize() – возвращает размер таблицы (значение поля `size`, т.е. тип `size_t`). Является константным методом.

~AVLtable() – деструктор. Освобождает ресурсы, выделенные для объекта данного класса.

Перегруженный оператор вывода: принимает ссылку на объект класса `ostream`,

ссылку на константный объект данного класса. Предназначен для отображения содержимого таблицы в стандартный поток вывода (консоль). Основное назначение — визуализация структуры дерева для проверки корректности балансировки при операциях вставки и удаления. Формат вывода: (ключ:данные),(ключ:данные) ... Порядок обхода — инфиксный: сначала выводится левое поддерево, затем корень, затем правое поддерево.

Описание класса HeshTable:

Данный класс является шаблонным и представляет собой реализацию хэш-таблицы. Описание полей (спецификатор доступа **private**):

- Структура record. Поля:
 - data - данные (шаблонного типа Tvalue)
 - key – объект класса std::string
 - flag_deleted – имеет тип int. Возможные значения: -1 – запись не использовалась, 0 – запись удалена, 1 – запись используется (занята)
 - ❖ **record()** – конструктор по умолчанию. Присваивает полю flag_deleted значение -1.
- table – объект класса std::vector, хранящего объекты типа record
- default_capacity – статическое константное поле. Имеет тип size_t, равно 500. Это начальная вместимость таблицы.
- capacity – имеет тип size_t. Это текущая вместимость таблицы (увеличиться в 2 раза, если пользователь использует метод **Resize()**)
- size – имеет тип size_t. Это текущий размер таблицы.
- count_operations – аналогично классам unordered_table, AVLtable.

Описание методов (спецификация доступа **private**):

hash(const std::string& key) – принимает ссылку на константный объект класса std::string. Возвращает тип size_t. Это функция хэширования. Назначение: используя переданный ключ, сформировать индекс ячейки table. Это обеспечивает быстрый доступ к данным по ключу.

repeat_hash(const size_t result_hash) – принимает константную переменную типа size_t. Метод предназначена для разрешения коллизий в хэш-таблице. Если при добавлении или поиске элемента вычисленный индекс уже занят другим ключом (коллизия), данная функция генерирует новый индекс для проверки следующей ячейки в таблице. Возвращает тип size_t.

find_ind(const std::string& key) – принимает ссылку на константный объект класса std::string. Использует методы **hash**, **repeat_hash** для поиска индекса ячейки таблицы по ключу. Если ячейки с данным ключом нет, то возвращается -1 (т.е. возвращается тип int).

Описание методов (спецификатор доступа **public**):

HashTable() – конструктор по умолчанию. Устанавливает полю size значение 0, полю capacity значение default_capacity. Для выделения памяти под хранение данных

вызывается метод `resize` вектора `table`, который устанавливает его размер равным значению `default_capacity`. Метод `resize` вызывает для каждой записи (ячейки вектора) конструкторы соответствующих полей. Поэтому у каждой записи изначально `flag_deleted` будет равен -1.

Resize() – возвращает тип `void`. Удваивает значение поля `capacity` и пересчитывает индексы всех используемых элементов (`flag_deleted` ячейки равен 1) с учётом нового размера. Для перераспределения данных используются методы хэширования **hash** и **repeat_hash**.

Назначение методов `Delete`, `Find`, `Insert`, `Extract` такое же, как и в классах `unordered_table`, `AVLtable` (принимаемые параметры и возвращаемые значения точно такие же). Разумеется, при этом используются методы **hash**, **repeat_hash**.

GetSize() – возвращает текущий размер таблицы (значение поля `size`, т.е. тип `size_t`). Является константным методом.

GetCapacity() – возвращает значение поля `capacity` (т.е. тип `size_t`).

Описание класса **Logging_operations**:

Данный класс является шаблонным и предназначен для записи количества операций в файл при выполнении определенных действий (поиск, вставка, удаление, извлечение) для всех типов таблиц.

Описание полей (спецификатор доступа **private**):

- Три массива, для хранения количества соответствующих операций. Индексы: 0 – класс `unordered_table`, 1 – `AVLtable`, 2 – `HashTable`.
 - `del` – массив из трех элементов типа `size_t`. Хранит число операций при выполнении `Delete` у всех таблиц.
 - `ins` – массив из трех элементов типа `size_t`. Хранит число операций при выполнении `Insert` у всех таблиц.
 - `find` – массив из трех элементов типа `size_t`. Хранит число операций при выполнении `Find` у всех таблиц.
 - `extr` – массив из трех элементов типа `size_t`. Хранит число операций при выполнении `Extract` у всех таблиц.
- Три массива, для хранения общего количества операций для каждого действия. Индексы массивов: 0 – общее число операций для `Delete`, 1 – общее число операций для `Insert`, 2 – общее число операций для `Find`, 3 – общее число операций для `Extract`.
 - `un_table_all_operations` – массив из 4 элементов типа `size_t` (для класса `unordered_table`)
 - `avl_table_all_operations` – массив из 4 элементов типа `size_t` (для класса `AVLtable`)

- `hash_table_all_operations` - массив из 4 элементов типа `size_t` (для класса `HashTable`)
- `name_file_without_dot_txt` – имя файла, в который будет происходить запись (объект класса `std::string`)

Описание методов (спецификатор доступа **public**):

Logging_operations() – конструктор по умолчанию. Задаёт имя файла, как “Logging operation.txt”. Также все ячейки массивов «all_operations» инициализируются нулями.

Logging_operations(const std::string& name_file_without_dot_txt) – конструктор инициализации. Принимает ссылку на константный объект класса `std::string` (передается имя файла). Если переданная строка оканчивается на “.txt”, то выбрасывается исключение. Иначе полю `name_file_without_dot_txt` присваивается результат конкатенации переданной строки и строки “.txt”. Все ячейки массивов «all_operations» инициализируются нулями.

Класс `Logging_operations` является дружественным для всех таблиц (`unordered_table`, `AVLtable`, `HeshTable`). Во всех таблицах есть `protected` метод:

get_count_operations() – возвращает значение поля `count_operations` (т.е. тип `size_t`). Является константным методом.

Перед выполнением определенного действия (поиск, вставка и т.д.) `count_operations` обнуляется (благодаря этому подсчет операций для конкретного действия является корректным)

В следующих методах класса `Logging_operations` применяется метод `get_count_operations`.

Delete_operations(const unordered_table<Tvalue>& t1, const AVLtable<Tvalue>& t2, const HashTable<Tvalue>& t3) – принимает ссылку на константный объект класса `unordered_table`, ссылку на константный объект класса `AVLtable`, ссылку на константный объект класса `HashTable`. Возвращает тип `void`. Сохраняет количество операций, затраченных на последнее удаление в каждой из трёх таблиц, во внутренний массив `del`. Кроме того, значения из массива `del` прибавляются к соответствующим значениям в массивах `un_table_all_operations`, `avl_table_all_operations` и `hash_table_all_operations` по индексу 0 (отвечающему за операцию удаления), что позволяет вести учёт общего числа операций за всё время работы программы.

Insert_operations(const unordered_table<Tvalue>& t1, const AVLtable<Tvalue>& t2, const HashTable<Tvalue>& t3) – аналогично методу `Delete_operations`, но в массивы записывается число операций, затраченное на вставку.

Find_operations(const unordered_table<Tvalue>& t1, const AVLtable<Tvalue>& t2, const HashTable<Tvalue>& t3) - аналогично методу `Delete_operations`, но в массивы записывается число операций, затраченное на поиск.

Extract_operations(const unordered_table<Tvalue>& t1, const AVLtable<Tvalue>& t2, const HashTable<Tvalue>& t3) - аналогично методу `Delete_operations`, но в массивы записывается число операций, затраченное на извлечение.

Writing_to_file(size_t mode) – принимает переменную типа size_t. Возвращает тип void. Возможные значения для переменной mode: 0 – операция Delete, 1 – операция Insert, 2 – операция Find, 3- операция Extract. Используется для записи в файл количества операций соответствующего действия. Формат записи (построчно):

1. Название операции: (Delete/Insert/Find/Extract)
2. Unordered_table: (число операций)
3. AVLtable: (число операций)
4. HashTable: (число операций)

Writing_end_work() – возвращает тип void. Используется для записи в файл общего количества операций для каждого действия каждого типа таблиц. Формат записи (построчно):

1. Общее число операций
2. Unordered_table:
3. Delete:
4. Insert:
5. Find:
6. Extract:
7. Общее число операций для данной таблицы:
После то же самое для AVLtable, HashTable (начиная со второй строки)

Clear() – возвращает тип void. Очищает содержимое файла.

В каждом из классов таблиц объявлены три дружественные функции:

- run_insert_experiment
- run_delete_experiment
- run_find_experiment

Каждая из данных функций использует метод get_count_operations. Данные функции используются при проведении экспериментов.

3.3. Описание алгоритмов

Описание алгоритмов класса `unordered_table`

Метод **Delete**:

Цикл `for` проходит по всей таблице. Если ключ i -го элемента вектора `table` совпадает с искомым, то i -ый элемент заменяется на последний. Последний элемент удаляется методом `pop_back`. После чего происходит выход из метода. Сложность алгоритма: $O(n)$, n – число элементов в векторе `table` (записей в таблице).

Метод **Find**:

Цикл `for` проходит по всей таблице. Если ключ i -го элемента вектора `table` совпадает с искомым, то возвращается указатель на поле `value` i -го элемента. Если ключ не был найден, то возвращается `nullptr`. Сложность алгоритма: $O(n)$

Метод **Insert**:

Если переданный ключ является пустой строкой, то выбрасывается исключения типа `std::runtime_error`. Вызывается метод `Find`. Если запись с данным ключом есть в таблице, то происходит выход из метода. Иначе в таблицу добавляется запись с переданными параметрами. Сложность алгоритма: $O(n)$.

Метод **Extract**:

Вызовом метода `Find` осуществляется поиск записи по ключу. Если запись не найдена (метод `Find` вернул `nullptr`), выбрасывается исключение `std::runtime_error`. Если запись найдена, создаётся копия данных из найденной ячейки. Запись удаляется из таблицы вызовом метода `Delete`. Созданная копия данных возвращается как результат работы метода. Сложность алгоритма: $O(n)$

Описание алгоритмов класса `AVLtable`

Метод **NewHeight**:

Если `node` не равен `nullptr`, то с помощью метода **getHeight** вычисляются высоты левого и правого потомков переданного узла. Итоговая высота текущего узла вычисляется как: $1 + \max(\text{высота левого потомка}, \text{высота правого потомка})$.

Метод **rightRotate**:

Применяется в случае, если баланс данной вершины меньше -1. Логика перестройки:

1. Запоминается левый потомок исходного узла (`l1`).
2. Запоминается правый потомок узла `l1` (`r2`).
3. Левый указатель исходного узла (`node`) теперь указывает на `r2` (после присвоения `node.left = r2` происходит вызов метода **NewHeight** для узла `node`)
4. Правый указатель узла `l1` теперь указывает на исходный узел `node` (после вызывается метод **NewHeight** для узла `l1`).
5. Из метода возвращается указатель `l1`.

Метод **leftRotate**:

Применяется в случае, если баланс данной вершины больше +1. Логика перестройки:

1. Запоминается правый потомок исходного узла (r1)
2. Запоминается левый потомок узла r1 (l2).
3. Правый указатель исходного узла указывает на l2 (после с помощью **NewHeight** пересчитывается высота узла node)
4. Левый указатель узла r1 указывает на исходный узел (после пересчитывается высота узла r1 с помощью метода NewHeight)
5. Из метода возвращается указатель r1.

Метод **Balancing**:

Пока стек не пуст:

1. Из стека извлекается указатель и запоминается во временную переменную current (после чего указатель удаляется из стека)
2. С помощью метода getHeight вычисляется высота current.
3. Если баланс current принимает корректные значения (от -1 до 1), то переходим к следующему указателю
4. Если баланс current равен -2, то запоминается указатель на левого потомка current (l).

Если баланс l меньше или равен 0, то для current вызывается метод rightRotate (малое правое вращение).

Иначе для l вызывается метод leftRotate. Левым потомком current становится l. После для current вызывается метод rightRotate. (большое правое вращение)

5. Если баланс current равен 2, то запоминается указатель на правого потомка current (r)

Если баланс r больше или равен 0, то для current вызывается метод leftRotate (малое левое вращение).

Иначе для указателя r вызывается метод rightRotate. Правым потомком current становится r. Для current вызывается метод leftRotate. (большое левое вращение)

Результаты конечных вращений (т.е. если большое вращение, то результат второго вращений; если малое, то результат первого) запоминаются в переменную new_current_root.

После new_current_root становилась правым или левым потомком предка current (в зависимости от ключа)

Сложность алгоритма: $O(h)$, где h – число элементов в path (т.к. дерево сбалансированное, то $h = \log_2 n$ (n – число элементов дерева))

Метод **print_node**:

Если указатель на текущий узел не равен nullptr, то метод вызывается для левого потомка данного узла. Если указатель равен nullptr, то происходит выход из рекурсии и ввод информации об узле в поток out. После метод вызывается для правого потомка текущего узла. Данный метод принимает параметр count. Данный параметр отвечает за количество запятых, которое нужно ввести в поток out. Сложность алгоритма: $O(k)$, где k – число узлов в поддереве узла node.

Метод **Find**:

1. Если поле root равно nullptr, то метод возвращает nullptr.
 2. Если проверка пройдена, то переменной current присваивается значение поля root.
 3. Цикл while работает пока переменная current не равна nullptr.
 - Если ключ current совпадает с искомым, то метод возвращает указатель на данные current.
 - Если ключ current лексикографически меньше искомого, то переменной current присваивается указатель на её правого потомка.
 - Иначе переменной current присваивается указатель на левого потомка.
 4. Если цикл while завершился (ключ не был найден), то метод возвращает nullptr.
- Сложность алгоритма: $O(\log_2 n)$, n – число узлов в дереве (т.к. данный класс — это AVL дерево)

Метод **Insert**:

1. Если переданный ключ является пустой строкой, то выбрасывается исключение типа `std::runtime_error`
 2. Если поле root равно nullptr, то происходит выделение памяти под новый узел с помощью new (новый узел создается с переданными параметрами). Полю root присваивается указатель на новый узел. После происходит выход из метода.
 3. Создается переменная path – объект класса `std::stack`, содержащего указатели на тип Node. Также создается переменная current, которой присваивается значение поля root.
 4. Цикл while работает, пока current не равна nullptr
 - В данном цикле осуществляется логика поиска переданного ключа. Если данный ключ найден, то происходит выход из метода. Также происходит заполнение объекта path.
 5. Вставка нового узла:
 - Из path извлекается указатель и запоминается в переменную parent (это предок нового узла)
 - Выделяется память под новый узел (новый узел создается с переданными параметрами)
 - Если ключ parent меньше переданного, то новый узел становится правым потомком parent.
 - Иначе новый узел становится левым потомком parent.
 6. Вызывается метод Balancing, в который передается переменная path.
- Сложность: в цикл while встроен поиск ключа (совместно с заполнением path). Данный цикл работает за $O(\log_2 n)$. В конце вызывается метод Balancing, который также работает за $O(\log_2 n)$. Итоговая сложность: $O(\log_2 n)$.

Метод **Delete**:

1. Если поле `root` равно `nullptr`, то происходит выход из метода.
2. Создается переменная `path` – объект класса `std::stack`, содержащего указатели на тип `Node`. Также создается переменная `current`, которой присваивается значение поля `root`.
3. Цикл `while` работает, пока `current` не равна `nullptr`
 - В данном цикле осуществляется логика поиска переданного ключа. Если данный ключ найден, то происходит выход из цикла. Также происходит заполнение объекта `path`.
4. Если `current` не равна `nullptr` (т.е. запись с данным ключом есть в таблице), то:
5. Создается переменная `parent` (указатель на тип `Node`), которой присваивается `nullptr`.
6. Если размер `path` не равен 1 (пользователь хочет удалить корень), то переменной `parent` присваивается указателю предка `current` (запись, которую нужно удалить)
7. Если `current` является листом (левый и правый потомки равны `nullptr`):
 - Если `parent` равен `nullptr`, то освобождается память под поле `root`. Полю `root` присваивается `nullptr`.
 - Если `current` – это левый потомок `parent`, то освобождается память, выделенная под левого потомка `parent` (после `parent.left` присваивается `nullptr`)
 - Если `current` – это правый потомок `parent`, то освобождается память, выделенная под правого потомка `parent` (после `parent.right` присваивается `nullptr`)
 - Из `path` удаляется последний элемент (т.е. `current`)
8. Если у удаляемой вершины есть один потомок (левый или правый):
 - В переменную `child` (указатель на тип `Node`) запоминается потомок удаляемой вершины
 - Если `parent` равен `nullptr`, то полю `root` присваивается `child`.
 - Если `current` – это левый потомок `parent`, то левым потомком `parent` становится `child`. Иначе `child` становится правым потомком `parent`.
 - Из `path` удаляется последний элемент (т.е. `current`)
 - Память, выделенная под удаляемую вершину, освобождается
9. Если у удаляемой вершины есть два потомка:
 - Создается переменная `maxLeftParent` (указатель на тип `Node`), которой присваивается значение `current`. Создается переменная `maxLeft` (указатель на тип `Node`), которой присваивается значение левого потомка `current`.
 - Цикл `while` работает, пока правый потомок `maxLeft` не равен `nullptr`. В данном цикле обновляются значения `maxLeft` и `maxLeftParent`. `maxLeft` примет значение самого правого узла (лексикографическое значение его ключа меньше, чем у удаляемой вершины, но самое большое в поддереве `maxLeft`), `maxLeftParent` – всё ещё предок `maxLeft`.
 - Если `maxLeftParent` равен `current` (т.е. максимальный ключ в левом поддереве – это ключ потомка удаляемой вершины), то:
 - Правому потомку `maxLeft` присваивается значение правого потомка `current`
 - Если `parent` равен `nullptr`, то полю `root` присваивается значение `maxLeft`
 - Если `current` – это левый потомок `parent`, то левым потомком `parent` становится `maxLeft`. Иначе `maxLeft` становится правым потомком `parent`.
 - Из `path` удаляется последний элемент (`current`).
 - В `path` помещается `maxLeft`.

- Иначе:
 - Правым потомком `maxLeftParent` становится левый потомок `maxLeft`
 - Правым потомком `maxLeft` становится правый потомок `current`
 - Левым потомком `maxLeft` становится левый потомок `current`
 - Если `parent` равен `nullptr`, то полю `root` присваивается значение `maxLeft`
 - Если `current` – левый потомок `parent`, то левым потомком `parent` становится `maxLeft`. Иначе `maxLeft` становится правым потомком `parent`.
 - Из `path` удаляется последний элемент (`current`). В `path` добавляются `maxLeft` и `maxLeftParent`.
- ❖ Данные, выделенные под удаляемую вершину, освобождаются.
- 10. Размер таблицы уменьшается на единицу. После в метод `Balancing` передается переменная `path`.
Сложность алгоритма: аналогично `Insert` $O(\log_2 n)$ (но, если у удаляемой вершины два потомка, то будет работать второй `while`, который также окажет влияние на сложность алгоритма)

Метод **Extract**:

1. Создается переменная `ptr` – указатель на тип данных `Node`. В него записывается результат метода `Find` (в метод `Find` передается тот же ключ, что и в `Extract`).
 2. Если `ptr` равен `nullptr` (записи с данным ключом в таблице нет), то выбрасывается исключение типа `std::runtime_error`.
 3. Если проверка пройдена, то разыменованный указатель присваивается переменной `result`. Запись с данным ключом удаляется из таблицы с помощью метода `Delete`.
 4. Метод возвращает переменную `result`.
- Сложность алгоритма:
- ❖ Сложность метода `Find`: $O(\log_2 n)$
 - ❖ Сложность метода `Delete`: $O(\log_2 n)$
- Итоговая сложность: $O(\log_2 n)$

~AVLtable() – деструктор:

1. Если поле `root` не равно `nullptr`, то:
2. Создаются два стека: `stack1`, `stack2`. Оба стека хранят указатели на тип `Node`.
3. В `stack1` помещается поле `root`.
4. Первый цикл `while` работает, пока `stack1` не пуст:
 - ❖ Извлекается верхний элемент из `stack1` (запоминается в переменную `current`).
 - ❖ Этот элемент помещается в `stack2` (в `stack1` его уже нет)
 - ❖ В `stack1` добавляются потомки узла `current`. Сначала добавляется правый потомок (если он есть (не равен `nullptr`)), а затем — левый (если он есть)
 - ❖ После завершения этого цикла стек `stack2` будет содержать все узлы дерева в порядке, обратном обходу. Это означает, что корневой узел окажется в самом низу стека, а листья — наверху.
5. Второй цикл `while` работает, пока `stack2` не пуст:
 - Из `stack2` последовательно извлекаются узлы.
 - Для каждого извлечённого узла (`current`) вызывается оператор `delete`, который освобождает занимаемую им память
 - После удаления указателю `current` присваивается значение `nullptr`, а счётчик `size` уменьшается на единицу.

6. После циклов полю `root` присваивается значение `nullptr`.
Сложность алгоритма: $O(n)$, где n – число узлов в дереве.

Перегруженный оператор вывода:

Использует метод **`print_node`**. Сложность алгоритма: $O(n)$, где n – число узлов в дереве.

Описание алгоритмов класса `HashTable`:

Метод `hash`:

1. Заводится переменная `res`, изначально равная 0. Она будет накапливать результат
2. Для каждого символа строки `key` выполняется:
 - Преобразование символа в целое число (его код)
 - Обновление значения `res` по формуле: $res = res \times 17 + (\text{int})key[i]$ (i – индекс символа строки)

*множитель 17 выбран произвольно (как простое число)
3. После обработки всех символов вычисляется остаток от деления `res` на вместимость таблицы (`capacity`): $res \% capacity$. Это гарантирует, что итоговый индекс попадёт в диапазон допустимых индексов вектора `table`.
Сложность алгоритма: $O(m)$, где m – число символов в строке.

Метод `repeat_hash`:

1. Метод принимает на вход индекс `result_hash`, полученный от основной хэш-функции.
2. К полученному индексу прибавляется единица, и результат делится по модулю на `capacity`.
3. Метод возвращает новый индекс, который указывает на следующую ячейку в массиве

Метод `find_ind`:

1. Вычисляется начальный индекс h с помощью метода `hash(key)`
2. Инициализируется счётчик шагов i , который ограничивает количество итераций, чтобы избежать заикливания.
3. Цикл `while` работает, пока i меньше `capacity`:
 - Если ячейка с индексом h не использовалась (`flag_deleted` равен -1), то возвращается -1 (т.к. это означает, что данной записи в таблице нет)
 - Если ключ записи (ячейки) с индексом h совпадает с искомым ключом и данная ячейка используется, то индекс h возвращается в качестве результата.
 - Иначе индекс h обновляется с помощью метода `repeat_hash` (также i увеличивается на единицу)
4. Если цикл `while` завершил работу, то возвращается -1.
Сложность алгоритма: в худшем случае $O(capacity)$, в среднем $O(1)$

Метод `Find`:

1. Если размер таблицы (поле `size`) равен 0, то возвращается `nullptr`.
2. С помощью метода `find_ind(key)` определяется индекс вектора `table` (h).

3. Если h не равен -1 , то метод возвращает указатель на данные соответствующей ячейки.
 4. Иначе возвращается `nullptr`.
- Сложность алгоритма: в среднем $O(1)$

Метод Insert:

1. Если поле `size` равно полю `capacity` (все ячейки вектора `table` заполнены), то метод завершает работу.
2. Если переданный ключ является пустой строкой, то выбрасывается исключение типа `std::runtime_error`.
3. Вычисляется начальный индекс h с помощью метода `hash(key)`
4. Инициализируется счётчик шагов i .
5. Инициализируется переменная `first_find_del` значением -1 .
6. Цикл `while` работает, пока i меньше `capacity`:
 - Если запись с данным ключом используется в таблице, то происходит выход из метода.
 - Если найдена первая удаленная запись с индексом h , то переменной `first_find_del` присваивается значение h
 - Если найдена ячейка с данным индексом, которая не использовалась и `first_find_del` принимает значение -1 , то в ячейку h заносится запись с переданными параметрами. Размер таблицы увеличивается на 1 , цикл прекращает работу.
 - Если ячейка с индексом h не используется и `first_find_del` больше -1 (т.е. была найдена удаленная запись), то в ячейку с индексом `first_find_del` заносится запись с переданными параметрами. Размер увеличивается на единицу, цикл прекращает работу.
 - Иначе h обновляет свое значение с помощью метода `repeat_hash`. Счётчик i увеличивается на 1 .

Сложность алгоритма: в среднем $O(1)$

Метод Delete:

1. Если текущий размер таблицы равен 0 , то происходит выход из метода
2. Вычисляется индекс h с помощью метода `find_ind(key)`
3. Если h равен -1 (запись с данным ключом не найдена), то происходит выход из метода.
4. Иначе полю `flag_deleted` ячейки массива `table` с индексом h присваивается 0 . Поле `size` уменьшается на 1 .

Сложность алгоритма: в среднем $O(1)$

Метод Extract:

1. Если поле `size` равно 0 , то выбрасывается исключение типа `std::runtime_error`.
2. Вычисляется индекс h с помощью метода `find_ind(key)`
3. Если h равен -1 , то выбрасывается исключение типа `std::runtime_error`.
4. Переменная `res` инициализируется данными из ячейки вектора `table` с индексом h .
5. Поле `flag_deleted` ячейки вектора с индексом h получает значение 0 .
6. В качестве результата возвращается переменная `res`.

Сложность алгоритма: в среднем $O(1)$

Метод **Resize**:

1. С помощью метода `std::move` данные из вектора `table` «переносятся» в вектор `old_table` (новая переменная)
 2. Переменная `old_capacity` инициализируется значением `capacity`.
 3. Поле `capacity` увеличивается вдвое.
 4. К вектору `table` применяется метод `resize` (`table.resize(capacity)`).
 5. Полю `size` присваивается 0.
 6. Цикл `for` проходит по всем ячейкам вектора `old_table`:
 - Если ячейка была активна (`flag_deleted` равен 1), то для неё вычисляется новый индекс `new_index` с помощью метода **hash** (если ячейка с данным индексом занята, то `new_index` обновляется с помощью метода **repeat_hash**)
 - В ячейку вектора `table` с новым индексом заносятся данные из вектора `old_table`.
 - Поле `size` увеличивается на 1.
- Сложность алгоритма: цикл `for` проходит по всем ячейкам старого массива ($O(\text{capacity})$). Временная переменная `old_table` хранит старые данные вектора `table`, т.е. оценка потребления памяти: $O(\text{capacity})$

4. Результаты экспериментов и тестирования

Во всех тестах в качестве данных для таблиц выступают полиномы из лабораторной работы №1.

Тесты Google Test для класса `unordered_table` (группа тестов: `test_unordered_table`; тест: `test_insert_find_delete`):

- В данном тесте проверяется корректность работы методов: `Insert`, `Find`, `Delete`, `Extract`, `GetSize`.
 - `Insert`: корректность вставки данных в таблицу, попытка вставить данные с ключом, который использовался ранее.
 - `Find`: корректность поиска данных в таблице, корректность возвращаемых значений (указатель на данные/`nullptr`), возможность изменить данные с помощью указателя.
 - `Delete`: корректность удаления записи из таблицы
 - `Extract`: корректность извлечения записи из таблицы, корректность выброса исключений
 - `GetSize`: корректность изменения размера таблицы после использования операций.

Тесты Google Test для класса `AVLtable` (единая группа тестов: `AVLtable`):

- **test_insert_find.** В данном тесте проверяется корректность работы методов `Insert` (вставка данных в таблицу, попытка вставить данные с ключом, который использовался ранее), `Find` (поиск данных в таблице, корректность возвращаемых значений, изменения внешнего полинома (переменной) не изменяет полином, ранее добавленный в таблицу и т.д.)
- **test_cout.** В данном тесте проверяется корректность хранения данных в таблице (применения левого/правого вращений) с помощью оператора вывода.
- **test_delete.** В данном тесте проверяется корректность работы метода `Delete`.
- **test_extract.** В данном тесте проверяется корректность работы метода `Extract` (корректность извлечения записи из таблицы, корректность выброса исключений).
- **test_destruct.** В данном тесте проверяется корректность работы деструктора (после его применения метод `GetSize` должен вернуть 0).

Тесты Google Test для класса `HashTable` (группа тестов: `test_hash_table`; тест: `test_operations`):

В данном тесте проверяется корректность работы методов `Insert`, `Find`, `Delete`, `Extract`, `GetSize` аналогично тестам для `unordered_table`, `AVLtable`. Также проверяется корректность метода `GetCapacity`.

Тесты Google Test для класса `Logging_operations` (группа тестов: `logging_op`; тест: `test_operations_file`):

В данном тесте проверяется корректность работы методов данного класса, а именно хранение и запись в файл количества действий при операциях `Insert`, `Find`, `Delete`, `Extract`

(т.е. проверяется корректность работы всех его методов, а также конструктора). Кроме того, проверяется корректность выброса исключений при указании некорректного названия файла в конструкторе.

Проведение экспериментов:

Цель проведения экспериментов: сравнить эффективность вставки, поиска и удаления элементов в таблицах при увеличении объема данных.

Основная работа происходит в функции main (файл test.cpp). В данной функции происходит генерация данных определенного объема с помощью функций **generate_key**, **generate_pol**. После с помощью функций **run_insert_experiment**, **run_find_experiment**, **run_delete_experiment** производился замер количества операций и времени для осуществления соответствующих действий (вставка, поиск, удаление) над определенным количеством данных для всех типов таблиц (в качестве данных используются полиномы из лабораторной работы №1). *Замер времени осуществляется с помощью объектов класса std::chrono (библиотека <chrono>). Результат хранится в секундах.

Результаты для объема данных: 400 элементов

Класс unordered_table:

- Метод Insert: 640 000 операций (время: 0.0029813 сек.)
- Метод Find: 642 000 операций (время: 0.0018082 сек.)
- Метод Delete: 203 000 операций (время: 0.001175 сек.)

Класс AVLtable:

- Метод Insert: 115 845 операций (время: 0.0028265 сек.)
- Метод Find: 21 286 операций (время: 0.0001607)
- Метод Delete: 78 717 операций (время: 0.0011764 сек.)

Класс HashTable:

- Метод Insert: 55 840 операций (время: 0.0002225 сек.)
- Метод Find: 51 040 операций (время: $9.34 \cdot 10^{-5}$ сек.)
- Метод Delete: 52 640 операций (время: $9.41 \cdot 10^{-5}$ сек.)

Результаты для объема данных: 900 элементов

Класс unordered_table:

- Метод Insert: 3 240 000 операций (время: 0.0100514 сек.)
- Метод Find: 3 244 500 операций (время: 0.0067786 сек.)
- Метод Delete: 1 019 250 операций (время: 0.0035951 сек.)

Класс AVLtable:

- Метод Insert: 291 037 операций (время: 0.0036252 сек.)
- Метод Find: 55 009 операций (время: 0.0003023 сек.)
- Метод Delete: 200 391 операция (время: 0.0029094 сек.)

Класс HeshTable:

- Метод Insert: 315 170 операций (время: 0.0007794 сек.)
- Метод Find: 304 370 операций (время: 0.0005476 сек.)
- Метод Delete: 307 970 (время: 0.0005591 сек.)

Результаты для объема данных: 1900 элементов

Класс unordered_table:

- Метод Insert: 14 440 000 операций (время: 0.0321795 сек.)
- Метод Find: 14 449 500 операций (время: 0.0266359 сек.)
- Метод Delete: 4 526 750 операций (время: 0.0135154 сек.)

Класс AVLtable:

- Метод Insert: 673 381 операция (время: 0.0073403 сек.)
- Метод Find: 130 148 операций (время: 0.0005548 сек.)
- Метод Delete: 461 373 операции (время: 0.0029094 сек.)

Класс HeshTable:

- Метод Insert: 1 348 410 операций (время: 0.0036694 сек.)
- Метод Find: 1 325 610 операций (время: 0.0023063 сек.)
- Метод Delete: 1 333 210 операций (время: 0.0036485 сек.)

Стоит отметить, что эксперименты проводились с ключами вида “key_0001, key_00002 и т.д.», объем данных почти был равен вместимости таблицы (это важно для класса HashTable)

На основе проведенных экспериментов можно сделать следующие выводы о сравнительной эффективности структур данных:

1. Неупорядоченная таблица (unordered_table) демонстрирует наихудшую производительность. Количество операций и время выполнения растут квадратично относительно объема данных. Эта структура неэффективна для больших объемов данных.
2. AVLtable (АВЛ дерево) обеспечивает стабильную производительность. Поиск является самой быстрой операцией, так как не требует перестройки дерева, в то время как вставка и удаление медленнее из-за необходимости балансировки.
3. Хэш-таблица (HashTable) показала наилучшие результаты по времени выполнения на малых и средних наборах данных. Для объемов N=400 и N=900 хэш-таблица значительно опережает другие структуры по скорости всех операций. Однако при объеме N=1900 наблюдается деградация производительности (рост количества операций). Это связано с тем, что эксперимент проводился при высокой загрузке таблицы, что привело к увеличению числа коллизий.

Общий вывод: для работы с данными большого объема наиболее эффективными структурами являются **хэш-таблица** и **сбалансированное дерево**. **Хэш-таблица** предпочтительна, если критически важна скорость доступа и структура не будет

перегружена. **Сбалансированное дерево** является универсальным решением, обеспечивающим логарифмическую скорость для всех операций независимо от распределения ключей и загрузки структуры.

5. Заключение

В ходе работы достигнуты следующие результаты:

- Реализован класс `unordered_table` (реализация неупорядоченной таблицы на основе динамического массива (т.е. вектора)))
- Реализован класс `AVLtable` (реализация упорядоченной таблицы на АВЛ дереве)
- Реализован класс `HeshTable` (реализация хэш-таблицы)
- Реализован класс `Logging operations`, позволяющий хранить число произведенных операций при выполнении определенного действия (поиск, вставка, удаление, извлечение) и записывать собранные данные в файл для всех типов таблиц.
- Написаны тесты Google Test, направленные на проверку корректности реализации классов `unordered_table`, `AVLtable`, `HeshTable`, `Logging operations`, работу методов данных классов при некорректных данных.
- Написан необходимый функционал для проведения экспериментов
- Проведена серия экспериментов по сравнительному анализу производительности структур данных при увеличении объема входных данных (400, 900 и 1900 элементов).
- Реализовано пользовательское консольное приложение, предоставляющее пользователю интерфейс для ввода полиномов, выбора необходимых операций и отображения результатов вычислений
- Расширено пользовательское консольное приложение из лабораторной работы №1. Приложение позволяет выполнять арифметические операции с полиномами и сохранять результаты в таблицы, а также управлять данными (добавлять, получать, удалять, извлекать)